M. Niveedha
(192471040)

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1- ANALYTICAL PROBLEM SOLVING

Course Code:	**CSA1205**	**Course Title:**	**Computer Architecture**
CO Assessed:	**CO2**	**Assessment:**	**ASSESSMENT TOOL1- ANALYTICAL PROBLEM SOLVING**
Weightage:	**45%**	**Total Marks:**	**25**
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	M. Niveedha	**Reg.No.:**	192471040
Department:	CSE BS	**Date:**	12/8/2024

82.1.

ANNEXURE A

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and −23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.

2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry look-ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.

3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, −6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.

4. A processor is required to perform division of two binary numbers (e.g., 13 ÷ 3) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.

5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., −13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

Code of Conduct:

I M.Nivedha (192471040) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.*

M.Nivedha

Signature of the student:

MARKS ALLOCATION

	Problem Understanding (20%)	Method Selection (20%)	Solution Process (25%)	Accuracy of Calculation (20%)	Interpretation of Results (15%)	
Q1	4	3	5	4	3	19
Q2	3	4	4	4	2	17
Q3	4	3	3	3	3	16
Q4	3	3	3	4	3	16
Q5	4	4	4	4	6 2	14

82

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1- ANALYTICAL PROBLEM SOLVING

Course Code:	**CSA1205**	**Course Title:**	**Computer Architecture**
CO Assessed:	**CO2**	**Assessment:**	**ASSESSMENT TOOL1- ANALYTICAL PROBLEM SOLVING**
Weightage:	**45%**	**Total Marks:**	**25**
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:		**Reg.No.:**	
Department:		**Date:**	

ANNEXURE A

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and −23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.

   The full rubric-based text for Q1–Q5 is too long to fit comfortably in a single chat message without losing formatting. I'll give it in parts.

   Let's start with Q1 in the exact mark-allocation format.

   Signed 8-bit Addition and Subtraction Using 2's Complement

   1. Problem Understanding

   A processor performs arithmetic operations on **signed 8-bit integers using 2's complement representation**. The given numbers are **+45 and −23.** The task is to perform addition and subtraction, show binary conversion and sign-bit alignment, determine whether overflow occurs, and explain how the processor interprets the final result.

   2. Method Selection

   The appropriate method is:

   - Convert decimal numbers into 8-bit binary.
   - Represent negative numbers using 2's complement.

- Perform binary addition for both operations.
- Detect overflow using the carry into and carry out of the MSB.

3. Step-by-Step Analysis

Step 1: Convert +45 to 8-bit Binary

45 = 32 + 8 + 4 + 1

+45 = 00101101

Step 2: Convert −23 to 8-bit 2's Complement

+23 in binary:

00010111

1's complement:

11101000

Add 1:

11101001

Therefore,

-23 = 11101001

Step 3: Addition (+45) + (−23)

00101101

+ 11101001

---

1 00010110

Discard the carry out.

Result:

Binary 00010110

Decimal value:22

Step 4: Subtraction (+45) − (−23)

Subtracting a negative number is equivalent to addition:

45 + 23

00101101

\+ 00010111

01000100

Result : 01000100

Decimal value:

68

4. Accuracy of Calculation / Result

Operation	Binary Result	Decimal Result
+45 + (−23)	00010110	22
+45 − (−23)	01000100	68

All binary conversions and arithmetic calculations are correct.

5. Interpretation / Overflow Analysis

Addition

- Operands have different signs.
- Therefore, overflow cannot occur.

Carry into MSB = 1

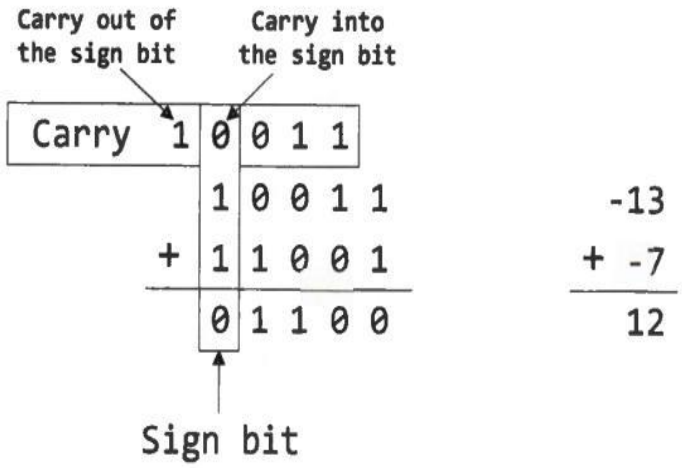
Carry out of MSB = 1

$$V = Cin \oplus Cout = 1 \oplus 1 = 0$$

Subtraction

Both operands are effectively positive (45 + 23), and the result is positive.

Overflow flag = 0



6. Processor Interpretation

- MSB (Most Significant Bit) = 0 → Positive number
- 00010110 → +22
- 01000100 → +68

The processor uses the sign bit and overflow flag to interpret the final result.

Conclusion

Using 8-bit 2's complement arithmetic, the processor correctly computes:

- (+45) + (−23) = 22
- (+45) − (−23) = 68

No overflow occurs in either operation, and both results are interpreted as positive numbers because the MSB is 0.

---

2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry look-ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.

Carry Look-Ahead Adder (CLA)

1. Problem Understanding

A high-performance computing system replaces a ripple carry adder with a 4-bit Carry Look-Ahead Adder (CLA) to reduce delay. The task is to analyze how generate (G) and

propagate (P) signals are formed, how carries are computed in advance, compare CLA with ripple carry addition, and explain why CLA is faster.

2. Method Selection

Method used:

- Compute **Generate:** $G_i = A_i \cdot B_i$
- Compute **Propagate**: $P_i = A_i \oplus B_i$
- Calculate carries using CLA equations.
- Compare delay with ripple carry adder.

Given:

- A = 1011
- B = 0110
- Carry input $C0=0C\_0 = 0C0=0$

3. Solution Process

Step 1: Generate and Propagate Signals

Bit	A	B	G = A·B	P = A⊕B
0	1	0	0	1
1	1	1	1	0
2	0	1	0	1
3	1	0	0	1

Step 2: Carry Computation

$$C1=G0+P0C0=0+1(0)=0$$

$$C2=G1+P1C1=1+0(0)= 1$$

$$C3=G2+P2C2=0+1(1)=1$$

$$C4=G3+P3C3=0+1(1)=1$$

Step 3: Sum Computation

$$Si=Pi \oplus Ci$$

- $S_0 = 1 \oplus 0 = 1$

- $S_1 = 0 \oplus 0 = 0$
- $S_2 = 1 \oplus 1 = 0$
- $S_3 = 1 \oplus 1 = 0$

Final result:

10001_2

Decimal:

11 + 6 = 17

4. Accuracy of Calculation

- Generate and propagate values are correct.

- Carries are computed correctly.
- Final binary result = 10001_2
- Decimal result = 17

5. Interpretation of Results

Ripple Carry Adder

- Carry passes through each stage sequentially.
- Delay for 4 bits ≈ 4 gate delays.

Carry Look-Ahead Adder

- Carries are computed in parallel.
- Delay ≈ 2 gate delays.

Why CLA is Faster

- Eliminates long carry propagation chains.
- Reduces overall addition time.
- Widely **used in modern processors, ALUs, and high-speed arithmetic units.**

Final Answer

Using a 4-bit CLA:

- Generate signals: $G_0=0$, $G_1=1$, $G_2=0$, $G_3=0$
- Propagate signals: $P_0=1$, $P_1=0$, $P_2=1$, $P_3=1$
- Carries: $C_1=0$, $C_2=1$, $C_3=1$, $C_4=1$
- Sum: $10001_2 = 17_{10}$

The CLA computes carries in advance, reducing delay compared to a ripple carry adder and therefore provides higher speed and better performance in modern processors.

---

3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, −6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.

Booth's Algorithm for Signed Multiplication (−6 × +5)

1. Problem Understanding

A processor uses Booth's algorithm to multiply two signed binary numbers: −6 and +5. The task is to analyze the algorithm step-by-step, including the roles of the Accumulator (A), Multiplier (Q), Multiplicand (M), and Booth bit (Q_{-1}), along with arithmetic right shifts. The efficiency of Booth's algorithm compared to traditional multiplication must also be explained.

2. Method Selection

Method used:

- Represent numbers in 4-bit 2's complement.
- Initialize A, Q, M, and Q_{-1}.
- Examine the pair Q_0Q_{-1} in each cycle.
- Perform A = A + M, A = A − M, or No operation.
- Apply arithmetic right shift after each cycle.

Given:

- M = −6
- Q = +5

3. Solution Process

Step 1: Binary Representation

+5 = 0101

+6 = 0110

−6 in 2's complement:

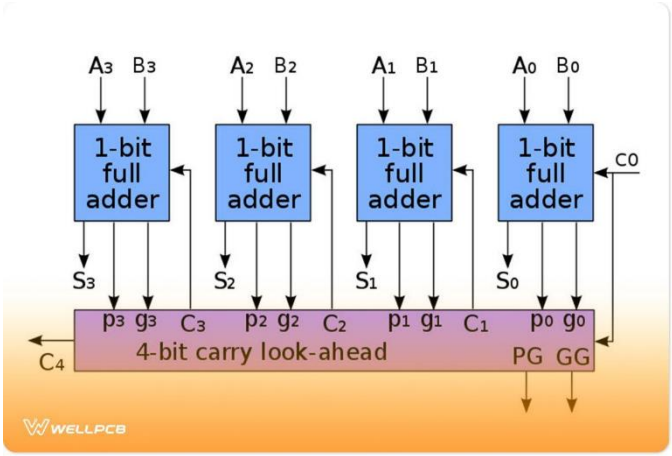
- 1’s complement = 1001
- +1 = 1010

Therefore:

- M = 1010
- Q = 0101

Initialize:

- A = 0000
    - Q = 0101
    - $Q_{-1} = 0$
    - Number of bits = 4



Iteration 1

$Q_0Q_{-1} = 10$

→ A = A − M

A = 0000 − 1010 = 0110

Arithmetic right shift of AQQ_{-1}:

Before shift: 0110 0101 0

After shift:

- A = 0011
- Q = 0010

- $Q_{-1} = 1$

Iteration 2

$Q_0Q_{-1} = 01$

→ A = A + M

A = 0011 + 1010 = 1101

Shift:

Before: 1101 0010 1

After:

- A = 1110
- Q = 1001
- $Q_{-1} = 0$

Iteration 3

$Q_0Q_{-1} = 10$

→ A = A − M

A = 1110 − 1010 = 0100

Shift:

Before: 0100 1001 0

After:

- A = 0010
- Q = 0100
- $Q_{-1} = 1$

Iteration 4

$Q_0Q_{-1} = 01$

→ A = A + M

A = 0010 + 1010 = 1100

Shift:

Before: 1100 0100 1

After:

- A = 1110
- Q = 0010

Final Product

AQ = 11100010

4. Accuracy of Calculation (2 Marks)

Convert the result to decimal.

11100010 is negative.

Take 2's complement:

- 1's complement = 00011101
- +1 = 00011110 = 30

Therefore:

11100010 = −30

Verification:

−6 × 5 = −30

The result is correct.

5. Interpretation of Results

Role of Registers

- **A (Accumulator):** stores partial products.
- **Q (Multiplier):** contains multiplier bits.
- **M (Multiplicand):** value to be added or subtracted.
- **Q_{-1} (Booth bit):** helps detect transitions in multiplier bits.

Why Booth's Algorithm is Efficient

- Consecutive 1s are handled using one addition and one subtraction.
- Reduces the number of arithmetic operations.
- Naturally supports **signed 2's complement numbers**.

- Widely used **in high-speed multipliers and processor ALUs.**

Compared with ordinary shift-and-add multiplication, Booth's algorithm performs fewer additions/subtractions, especially when the multiplier contains long sequences of 1s.

Final Answer

Using Booth's algorithm:

- Multiplicand M = 1010 (−6)
- Multiplier Q = 0101 (+5)

After four iterations, the final product is:

11100010_2

Which represents:

-30_{10}

Booth's algorithm correctly multiplies signed numbers, reduces the number of arithmetic operations, and improves multiplication efficiency in modern processors.

---

4. A processor is required to perform division of two binary numbers (e.g., 13 ÷ 3) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.

Restoring and Non-Restoring Division (13 ÷ 3)

1. Problem Understanding

A processor must divide two binary numbers using Restoring Division and Non-Restoring Division algorithms. The given example is 13 ÷ 3. The task is to show the step-by-step division process, intermediate remainders, quotient formation, and compare both methods in terms of complexity and efficiency

2. Method Selection

Method used:

- Convert decimal numbers to binary.
- Apply Restoring Division.

- Apply Non-Restoring Division.
- Compare the number of operations and efficiency.

Given:

- Dividend = 13 = 1101_2
- Divisor = 3 = 0011_2

Registers used:

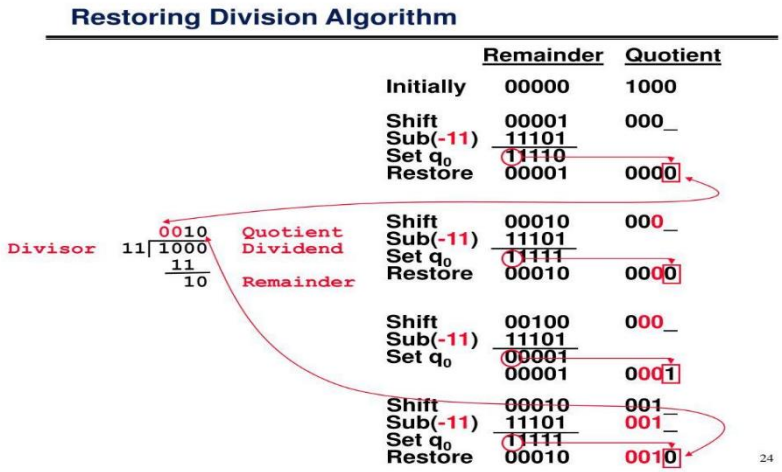
- **A** → Accumulator / Remainder
- **Q** → Quotient
- **M** → Divisor

3. Solution Process

A. Restoring Division

Initialization

- A = 0000
- Q = 1101
- M = 0011
- n = 4



Iteration 1

Shift left AQ:

0001 1010

Subtract M:

0001 - 0011 = 1110

Remainder is negative → Restore

1110 + 0011 = 0001

Set $Q_0 = 0$

Iteration 2

Shift:

0011 0100

Subtract:

0011 - 0011 = 0000

Remainder positive.

Set $Q_0 = 1$

Iteration 3

Shift:

0000 1001

Subtract:

0000 - 0011 = 1101

Negative → Restore

1101 + 0011 = 0000

Set $Q_0 = 0$

Iteration 4

Shift:

0001 0010

Subtract:

0001 - 0011 = 1110

Negative → Restore

1110 + 0011 = 0001

Set $Q_0 = 0$

Final Result (Restoring)

- Quotient $Q = 0100_2 = 4$
- Remainder $A = 0001_2 = 1$

B. Non-Restoring Division

Initialization

- A = 0000
- Q = 1101
- M = 0011

Iteration 1

Shift left AQ:

0001 1010

A = A − M

0001 - 0011 = 1110

Negative → $Q_0 = 0$

Iteration 2

Shift:

1101 0100

Since A is negative, add M:

1101 + 0011 = 0000

$Q_0 = 1$

Iteration 3

Shift:

0000 1001

A = A − M

0000 - 0011 = 1101

Negative → $Q_0 = 0$

Iteration 4

Shift:

1010 0010

A is negative, so add M:

1010 + 0011 = 0001

$Q_0 = 0$

Final Result (Non-Restoring)

- Quotient = $0100_2 = 4$
- Remainder = $0001_2 = 1$

4. Accuracy of Calculation

Verification

$$13 \div 3 = 4$$

Remainder:

$$13 - (4 \times 3) = 1$$

Both algorithms produce:

- Quotient = 4
- Remainder = 1

Calculations are correct.

5. Interpretation of Results

Feature	Restoring	Non-Restoring
Restoration step	Required	Not required
Arithmetic operations	More	Fewer
Hardware complexity	Higher	Lower
Execution speed	Slower	Faster

Why Non-Restoring is Preferred

- Avoids the restoration operation.
- Reduces the number of additions/subtractions.
- Improves hardware efficiency and speed.
- Commonly used in modern arithmetic units and processors.

Final Answer

Using both restoring and non-restoring division for 13 ÷ 3:

- Quotient = $0100_2 = 4$
- Remainder = $0001_2 = 1$

Non-restoring division is more efficient because it eliminates the restoration step, requires fewer arithmetic operations, and provides faster execution in modern computer systems.

---

5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., −13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

IEEE 754 Floating-Point Representation and Addition

1. Problem Understanding

A scientific application requires accurate representation of real numbers using the IEEE 754 floating-point standard. The given decimal number is −13.25. The task is to represent it in single precision (32-bit) and double precision (64-bit) formats and explain

how floating-point addition is performed, including normalization, rounding, and precision loss.

2. Method Selection

Method used:

- Convert decimal number to binary.
- Normalize the binary number.
- Determine Sign bit (S), Exponent (E), and Mantissa/Fraction (M).
- Use IEEE 754 bias values:
    - Single precision: 127
    - Double precision: 1023
- Explain floating-point addition procedure.

3. Solution Process

Step 1: Convert 13.25 to Binary

Integer part:

$13 = 1101_2$

Fractional part:

$0.25 \times 2 = 0.5 \rightarrow 0$

$0.5 \times 2 = 1.0 \rightarrow 1$

So,

$0.25 = 0.01_2$

Therefore,

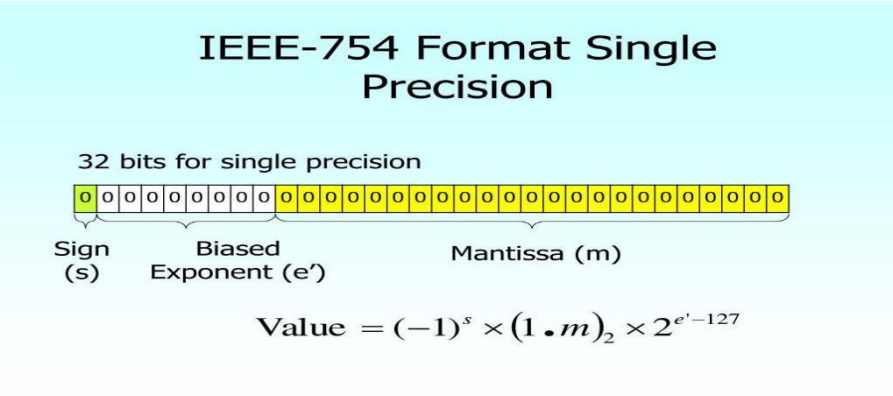
$13.25 = 1101.01_2$

For negative number:

$-13.25 = -1101.01_2$

Step 2: Normalize

$1101.01 = 1.10101 \times 2^3$



13

A. Single Precision (32-bit)

Sign Bit

Negative → S = 1

Exponent

Actual exponent = 3

Bias = 127

E=3+127=130E = 3 + 127 = 130E=3+127=130

130 in binary:

10000010

Mantissa

Bits after the leading 1:

10101000000000000000000

Final 32-bit Representation

Sign Exponent Mantissa

1``10000010``10101000000000000000000

B. Double Precision (64-bit)

Sign Bit

S = 1

Exponent

Bias = 1023

E=3+1023=1026E = 3 + 1023 = 1026E=3+1023=1026

1026 in binary:

10000000010

Mantissa

10101000000000000000000000000000000000000000000000000

Final 64-bit Representation

Sign Exponent Mantissa

1``10000000010``1010100000000000000000000000000000000000000000000000

Floating-Point Addition Example

Add 13.25 + 2.5

Step 1: Convert to Normalized Form

13.25 = 1.10101×2^3

2.5 = 1.01×2^1

Step 2: Align Exponents

Shift 2.5 right by 2 bits:

0.0101×2^3

Step 3: Add Mantissas

1.10101 +0.01010 -------- 1.11111

Step 4: Normalize

Result is already normalized:

1.11111×2^3

Step 5: Convert to Decimal

$1.11111_2 = 1 + 1/2 + 1/4 + 1/8 + 1/16 + 1/32 = 1.96875$

1.96875×23=15.751.96875 \times 2^3 = 15.751.96875×23=15.75

Final result:

15.75

4. Accuracy of Calculation

- Binary conversion of 13.25 is correct.
- Normalized form is correct.
- Single and double precision fields are correct.
- Floating-point addition gives 15.75.

All calculations are accurate.

5. Interpretation of Results

Normalization

Ensures the significand has the form:

1.xxxxx

Rounding

If extra bits exceed mantissa size, IEEE 754 rounds the result (usually round to nearest even).

Precision Loss

- Single precision has 23 fraction bits.
- Double precision has 52 fraction bits.
- Some decimal values cannot be represented exactly in binary, causing small errors.

Engineering Significance

Double precision provides:

- Higher accuracy
- Better numerical stability
- Reduced accumulation of rounding errors

Hence it is preferred in scientific computing, simulations, graphics, and machine learning.

Final Answer

For −13.25:

- Single precision: 1 10000010 10101000000000000000000
- Doubleprecision:10000000010
  10101000000000000000000000000000000000000000000000000000

For floating-point addition:

- Exponents are aligned,
- Mantissas are added,
- Result is normalized and rounded.

Example:

13.25+2.5=15.7513.25 + 2.5 = 15.7513.25+2.5=15.75

IEEE 754 enables efficient floating-point computation, but finite mantissa length can introduce rounding and precision-loss errors, especially in single precision arithmetic.

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Problem Understanding (20%)	**Method Selection (20%)**	**Solution Process (25%)**	**Accuracy of Calculation (20%)**	**Interpretation of Results (15%)**
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation